

Question 1 of 60

Flag

Scenario: Multi-Agent Research Sy

After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

- ☐ **A** The subagents need to share a single API connection to enable automatic context sharing between invocations.
- ☐ **B** The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.
- ☐ **C** The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.
- ☒ **D** The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

54 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60

Question 2 of 60

Flag

Scenario: Multi-Agent Research

The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information typically flow between these two specialized subagents?

- ☐ **A** The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.
- ☐ **B** Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.
- ☐ **C** The web search agent directly invokes the document analysis agent, passing the discovered sources as parameters.
- ☒ **D** The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

54

46 47 48 49 50 51 52 53 54 55 56 57 58 59 60

Question 3 of 60

Flag

Scenario: Multi-Agent Research S

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

- ☒ **A** Update the synthesis agent to render each content type appropriately—financial data as tables, news as prose
- ☐ **B** Standardize all subagent outputs to JSON with fields for `claim`, `evidence`, `source`, and `confidence`
- ☐ **C** Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation
- ☐ **D** Standardize all subagent outputs to prose summaries with inline citations

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations—the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

- ☐ **A** Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.
- ☐ **B** Have the report generator query the web search agent to re-locate sources for claims in the final report.
- ☐ **C** Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.
- ☒ **D** Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).

You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization—the synthesis agent attempts web searches, and the report generator tries to analyze documents. What is the primary cause of this poor tool selection behavior?

- ☐ **A** The tool definitions consume too much context window space, leaving insufficient room for task content.
- ☒ **B** The agents' role descriptions in their system prompts conflict with having access to tools outside that role.
- ☐ **C** The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.
- ☐ **D** Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach providing the synthesis subagent with the information it needs?

- ☐ **A** Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results
- ☐ **B** Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks
- ☒ **C** Include the complete findings from both subagents directly in the synthesis subagent's prompt
- ☐ **D** Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps—for instance, noting "the retrieved sources discuss API authentication but lack details on token refresh patterns"—but under the current strict pipeline, this insight isn't actionable since search has already completed. What's the most effective architectural change?

- ☐ **A** Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.
- ☐ **B** Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.
- ☐ **C** Add a research planning agent before the search phase that decomposes topics into specific sub-questions.
- ☒ **D** Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.

Production monitoring shows that follow-up queries like "summarize what we learned about market trends" consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research. What's the most effective way to improve response time for these follow-up summaries?

- ☐ **A** Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.
- ☒ **B** Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analysis.
- ☐ **C** Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings.
- ☐ **D** Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand.

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

- ☐ **A** Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.
- ☐ **B** Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.
- ☐ **C** Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.
- ☒ **D** Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate—it generates messages like "I'll ask the web search agent to find sources on this topic"—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

- ☐ **A** The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.
- ☐ **B** The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.
- ☒ **C** The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.
- ☐ **D** Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- ☐ **A** Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents
- ☒ **B** Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis
- ☐ **C** Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations
- ☐ **D** Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

- ☐ **A** Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.
- ☒ **B** Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.
- ☐ **C** Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.
- ☐ **D** Pass the full accumulated context from all prior agents.

The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

- ☒ **A** Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.
- ☐ **B** Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities.
- ☐ **C** Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category.
- ☐ **D** Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure."

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

- ☐ **A** Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.
- ☒ **B** Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.
- ☐ **C** Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.
- ☐ **D** Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate \$50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates \$35B ($\pm$ \$7B, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the market is estimated at \$35B-\$50B depending on factors." What systematic approach best addresses this?

- ☐ **A** Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator.
- ☒ **B** Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological context.
- ☐ **C** Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.
- ☐ **D** Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calibrated confidence.

Users report that responses feel repetitive across turns—each message begins with phrases like "Certainly!" or "I'd be happy to help!" even deep into conversations. You want responses to feel more natural without these repetitive openers. What's the most effective approach?

- ☐ **A** Append a partial assistant message with a direct response opening that the model will continue from
- ☐ **B** Lower the temperature parameter to make response openings more deterministic and less variable
- ☐ **C** Implement post-processing to detect and strip common greeting phrases from response beginnings
- ☒ **D** Add system prompt instructions specifying phrases to avoid, such as "Never begin responses with 'Certainly' or similar affirmations"

After three months of weekly sessions, your conversation history has grown to 85,000 tokens. When users ask "What did we conclude about the theme of isolation?", the assistant provides generic literary analysis rather than referencing the group's specific insights from earlier sessions. Discussions often build on previous meetings' conclusions, so maintaining narrative flow is important. What's the most effective approach?

- ☒ **A** Implement progressive summarization where older conversation blocks are replaced with concise summaries that explicitly extract key conclusions, decisions, and recurring themes, while keeping recent exchanges verbatim.
- ☐ **B** Add structured XML tags to mark significant discussion conclusions throughout the conversation history.
- ☐ **C** Implement rolling window truncation to keep only the most recent 25,000 tokens.
- ☐ **D** Use semantic embedding to index the full conversation history and retrieve only relevant past exchanges for each user query, replacing the linear conversation format with retrieved chunks.

During a conversation about order tracking, your external system receives a webhook indicating the user's package has shipped. The user is actively chatting and will likely send a follow-up message soon. You want the assistant to naturally incorporate this status change in its next response. What's the most effective approach?

- ☒ **A** Add the current shipping status to the system prompt before the next API call.
- ☐ **B** Configure the assistant to call a `get_order_status` tool at the start of every response.
- ☐ **C** Immediately send an API request with the update as a synthetic user message, generating an unsolicited assistant response.
- ☐ **D** Append the status update as a prefix to the next user message before calling the API.

Your music discovery assistant should consistently maintain an enthusiastic tone, explain its reasoning for each recommendation, and ask clarifying questions to better understand user preferences. You want this behavior to persist reliably across all user interactions. Where should you define these behavioral guidelines?

- ☒ **A** In the system prompt
- ☐ **B** Prepend to each user message before sending to the API
- ☐ **C** In the first assistant message, instructing Claude to follow these guidelines going forward
- ☐ **D** In environmental variables that your application passes to the API client

Your conversation history includes two types of content: persistent story elements (character backgrounds, plot structure, world rules) that must remain consistent throughout, and extensive brainstorming discussion that's mostly ephemeral. After 40+ turns, you're hitting context limits and users report the assistant "forgets" established character traits, breaking narrative consistency. Which approach best ensures persistent story elements remain available to the model while reclaiming context space?

- ☐ **A** Apply a sliding-window approach keeping only the most recent 25 turns, relying on the model to infer earlier context from recent discussion flow.
- ☐ **B** Summarize the entire conversation history into a condensed synopsis every 20 turns, replacing the full history to free up tokens.
- ☒ **C** Separate persistent story elements into a retained "story bible" section at context start, applying trimming or summarization only to brainstorming discussion.
- ☐ **D** Store all history in a vector database and retrieve semantically similar passages for each new message, replacing conversation history with retrieved chunks.

Users report that during extended conversations, the AI loses track of specific topics, examples, and preferences they mentioned earlier in the session. Your current implementation uses a sliding window that keeps only the most recent 25 message pairs to stay within context limits. What's the most effective approach to maintain awareness of earlier conversation content while managing context size?

- ☐ **A** Increase the window size to 50 message pairs to retain more conversation history before truncation.
- ☐ **B** Implement vector similarity search over the full conversation history, retrieving relevant past messages for each user query.
- ☒ **C** Replace the sliding window with a hybrid approach: summarize older messages while keeping recent messages verbatim.
- ☐ **D** Add a separate API call each turn to summarize messages being dropped, prepending this running summary to the conversation.

Your fitness coaching assistant uses a system prompt with detailed conditional logic: "If the user mentions being a beginner, provide step-by-step form instructions. If they use terms like 'progressive overload' or 'superset', respond concisely. If they ask about injury history, always recommend consulting a physician." During evaluation, you find the assistant correctly adapts to explicit expertise declarations but struggles when users don't clearly state their level—often defaulting to overly detailed responses regardless of contextual cues like technical terminology. Which change to the system prompt would most directly address this failure to pick up on implicit expertise signals?

- ☐ **A** Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses.
- ☒ **B** Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about injury consultations.
- ☐ **C** Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message.
- ☐ **D** Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced."

Your home renovation planning assistant uses a system prompt defining an expert contractor persona with specific guidelines: always ask about budget, suggest alternatives at multiple price points, and confirm timeline requirements. During testing, responses follow these guidelines for turns 1-4, but by turn 7, the assistant gives generic advice without asking about budget or timeline. The conversation totals only 2,500 tokens. What is the most likely cause?

- ☒ **A** The system prompt is only sent with the first API request.
- ☐ **B** The model's attention on system prompt instructions naturally weakens as turns accumulate.
- ☐ **C** The assistant's accumulated responses are diluting the system prompt's influence.
- ☐ **D** System prompts only establish initial behavior and don't persist across all turns.

Users frequently refine their search criteria mid-conversation. You notice a pattern: when users say things like "Actually, let's raise the budget to \$650K" or "I'd prefer a condo now instead of a house," the assistant sometimes continues referencing the original preferences in later responses—even though the updates are clearly present in the conversation history. Context usage is at 35% capacity. Which solution most reliably ensures the model uses the current preferences?

- ☐ **A** Implement conversation pruning to remove turns containing outdated preferences, ensuring only current ones remain in context.
- ☐ **B** Include few-shot examples showing the assistant correctly acknowledging and applying preference changes in responses.
- ☐ **C** Add system prompt instructions emphasizing that the model should always prioritize the most recently stated preferences over earlier ones.
- ☒ **D** Maintain a structured state object with current preferences, update it on changes, and include it in each request.

Your conversational assistant frequently generates multiple clarifying questions when users make ambiguous requests. When a user asks "Can you help me with the report?", the assistant responds: "I'd be happy to help! Could you tell me: 1) Which report? 2) What kind of help—drafting, reviewing, or formatting? 3) What's your deadline?"

User analytics show a 40% conversation abandonment rate after these multi-question responses. What's the most effective way to reduce friction while appropriately handling ambiguity?

- ☐ **A** Add a preprocessing step using a smaller model to classify request ambiguity on a 1-5 scale, routing high-ambiguity requests to a clarification dialog and low-ambiguity requests directly to the assistant.
- ☐ **B** Limit the assistant to one clarifying question per turn, using conversation history to accumulate answers over multiple exchanges rather than requesting everything upfront.
- ☐ **C** Create a lookup table of common request patterns with predefined default interpretations, having the assistant respond with those defaults without stating the assumptions made.
- ☒ **D** Modify the system prompt to instruct the assistant to make reasonable assumptions from available context, state those assumptions explicitly, and offer to adjust if the interpretation is wrong.

A new user's first message is "Set up my focus music." This could mean configure preferences, create a playlist, or play music immediately. Your system supports all three actions. What's the effective approach?

- ☐ **A** Create a new "Focus" playlist with curated tracks and notify the user it's ready.
- ☐ **B** Ask one clarifying question about action type: play now or configure for later.
- ☐ **C** Play popular focus tracks immediately and let the user redirect if needed.
- ☒ **D** Start preference configuration by asking about genres, tempo, and artists they prefer for focus.

After 30+ turns, your conversational assistant shows noticeably slower responses and occasionally produces less coherent outputs. Investigation reveals: (1) average conversations reach 50 tokens by turn 35, (2) production logs show 94% of user messages only reference the previous 3-5 exchanges, (3) the 6% of queries referencing earlier context typically ask about information user could easily re-state. Your goal is to improve response speed and quality while maintaining good user experience. What's the most effective approach?

- ☐ **A** Enable prompt caching and continue sending the complete conversation history, using cached prefixes to reduce per-request costs while preserving all context.
- ☐ **B** Implement a sliding window keeping only the system prompt and last 8-10 turns. When users reference earlier context, acknowledge the limitation and ask them to re-state the relevant information.
- ☐ **C** Build a retrieval system that stores all conversation turns and uses semantic search to pull in relevant historical context only when the current query appears to reference past information.
- ☒ **D** Implement a summarization layer that progressively compresses older conversation turns into a running summary while keeping the most recent 5-6 turns verbatim, maintaining full historical context in condensed form.

You're implementing a feature where users refine their playlist preferences through multiple conversation turns. After deploying, you notice Claude's responses don't reflect what users said in the same conversation—for example, a user says they love jazz, but two messages later Claude asks what genres they enjoy. What is the most likely cause?

- ☐ **A** The model's context window has been exceeded by the conversation length
- ☐ **B** The Claude API requires a `session_id` parameter that you haven't configured
- ☐ **C** Claude requires a vector database connection to maintain conversation memory
- ☒ **D** Your application isn't including prior messages in the messages array

During QA testing, you notice that Claude follows your system prompt guidelines consistently in the first 10-15 turns, but by turn 25-30, responses begin deviating—using informal tone when formality was specified, occasionally skipping required formatting, or providing information types the guidelines restrict. Conversation length is well within context limits (typically 30,000 tokens out of 200,000 available). What's the most effective approach to maintain consistent behavior throughout extended conversations?

- ☒ **A** Implement post-response validation that regenerates each response until it conforms to the specified guidelines.
- ☐ **B** Move behavioral guidelines from the system prompt into the first user message.
- ☐ **C** Automatically start a new conversation after 20 turns, passing a summary of the prior context to maintain continuity.
- ☐ **D** Insert user-role messages that reinforce critical guidelines at natural conversation breakpoints, especially before complex requests.

Over several turns discussing investment strategy, a user has stated: "I have a very low risk tolerance—I can't handle seeing my portfolio drop" and later, "I want to maximize my returns—I've seen my friends do well in crypto and tech stocks." They now ask "What should I invest in?" Which approach best ensures the resulting recommendation aligns with the user's actual priority?

- ☐ **A** Recommend a balanced portfolio that moderates both preferences without explicitly addressing the conflict.
- ☐ **B** Proceed with the most recently stated preference.
- ☐ **C** Provide separate recommendations for both scenarios (conservative and growth-oriented) without clarifying which the user prefers, and let them choose.
- ☒ **D** Surface the contradiction and ask the user to clarify which matters more—capital preservation or growth potential—before recommending.

An engineer submits two requests:

- Request A: "Rename the `getUserData` function to `fetchUserProfile` everywhere it's used."
 - Request B: "Improve error handling throughout the data processing module—add try/catch blocks, meaningful error messages, and ensure failures don't silently corrupt data."
- For which request does specifying an explicit multi-phase workflow (such as analyze → propose → implement with review) most improve outcome quality?

- ☐ **A** Request A, the function rename task
- ☒ **B** Request B, the error handling task
- ☐ **C** Both requests benefit equally
- ☐ **D** Neither request benefits significantly

You're building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function like `eval()` across a large codebase, which tool should your agent use for content search?

- ☒ **A** Use Grep to search for the pattern `"eval("` across all files in the codebase.
- ☐ **B** Read the project's main entry file and follow import statements to trace where `eval` might be used.
- ☐ **C** Use Bash to run `ls -R | grep eval` to recursively list files containing `eval`.
- ☐ **D** Use Glob with a pattern like `**/*eval*` to find files, then Read each matching file.

An engineer sees an unfamiliar error message "SYNC_CONFLICT: entity version mismatch detected" in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

- ☒ **A** Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.
- ☐ **B** Use Glob to find files in directories commonly associated with error handling (such as `errors/`, `exceptions/`, or `handlers/`) across all services, then Read each matching file.
- ☐ **C** Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service directory.
- ☐ **D** Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.

An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't specified which modules to prioritize. How should the agent decompose this open-ended task?

- ☐ **A** Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically.
- ☐ **B** Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or business importance.
- ☐ **C** Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for every function before beginning.
- ☒ **D** Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered.

Your productivity agent connects to three MCP servers: an issue tracker (search_issues, get_issue, create_comment), a documentation wiki (search_docs, get_page, list_spaces), and a data explorer (run_query, get_schema, list_databases). When engineers ask cross-system questions like "What database tables are affected by the authentication refactor in PROJ-1234?", monitoring shows the agent makes 8-10 sequential tool calls, frequently issues exploratory calls because it lacks visibility into what content each server contains, and exhausts context space before completing complex investigations. What architectural change best leverages MCP capabilities to address these issues?

- ☐ **A** Add a prepare_investigation tool to each server that accepts a natural language question and returns relevant content summaries
- ☒ **B** Expose each server's content catalog as MCP resources—issue summaries, documentation hierarchy, database schemas
- ☐ **C** Add an orchestrator that routes questions to a single server based on keywords
- ☐ **D** Consolidate all three servers into a unified MCP server with cross-referencing capabilities

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module). What exploration strategy will most reliably identify all callers?

- ☐ **A** Use Grep to search for the function's original name across the codebase.
- ☐ **B** Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.
- ☐ **C** Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.
- ☒ **D** Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.

During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about code structure it discussed earlier. Engineers report having to repeat context about modules they've already explored. What's the most effective approach to address this?

- ☐ **A** Create summaries of all source files before exploration begins, loading only these compressed representations into context.
- ☐ **B** Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.
- ☒ **C** Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.
- ☐ **D** Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.

An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's the most effective next step for building understanding while managing context constraints?

- ☒ **A** Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.
- ☐ **B** Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.
- ☐ **C** Use Glob to find files matching common caching patterns (`cache.py`, `caching/`), prioritize the largest files by reading them first, then check smaller files for gaps.
- ☐ **D** Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its `old_string` parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?

- ☒ **A** Use Read to load the file, add the function at the appropriate location, then Write the updated file
- ☐ **B** Use Edit with an extremely long `old_string` capturing 30+ lines of context to guarantee uniqueness
- ☐ **C** Use Bash to append the function definition to the end of the file using heredoc syntax
- ☐ **D** Use Edit's `replace_all` parameter to target a common pattern and embed the new function in the replacement text

Your agent has analyzed a complex service module—reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

- ☐ **A** Start two fresh sessions, having each re-read the relevant source files before beginning.
- ☒ **B** Resume the analysis session with `fork_session` enabled, creating a separate branch for each testing strategy.
- ☐ **C** Export the analysis session's key findings to a file, then create two new sessions that reference this file.
- ☐ **D** Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800 files across multiple services. What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- ☒ **A** Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.
- ☐ **B** Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.
- ☐ **C** Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.
- ☐ **D** Read all files containing "auth", "login", "permission", or "token" in their content or filename.

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth—having the agent propose specific code changes for each—before deciding which to implement. What's the most effective way to structure this exploration?

- ☒ **A** Use `fork_session` to create two branches from yesterday's analysis, exploring one approach in each fork.
- ☐ **B** Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.
- ☐ **C** Resume yesterday's session and explore both approaches sequentially within the same conversation thread.
- ☐ **D** Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to "create a new branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

- ☐ **A** You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- ☒ **B** Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.
- ☐ **C** The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.
- ☐ **D** The agent automatically selects the most relevant server based on the request and loads only that server's tools.

Engineers frequently ask the agent to cross-reference code changes with Jira tickets during reviews—checking ticket descriptions, acceptance criteria, and recent comments. This currently requires manually copy-pasting content into conversations. The team wants the agent to access this standard Jira ticket data directly. What's the most effective approach?

- ☒ **A** Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.
- ☐ **B** Build a custom MCP server wrapping Jira's API with tools designed specifically for this team's code review workflow.
- ☐ **C** Use the Bash tool with `curl` to call Jira's REST API, including authentication headers and parsing JSON responses inline.
- ☐ **D** Export Jira tickets to markdown files in the repository that the agent accesses using the Read tool.

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration. What's the most effective approach?

- ☐ **A** Resume the subagent from its previous transcript and inform it about the renamed functions.
- ☐ **B** Resume the subagent from its previous transcript without mentioning the changes—the architecture understanding remains valid.
- ☒ **C** Launch a fresh subagent with a summary of prior findings.
- ☐ **D** Launch a fresh subagent and include the prior transcript in the initial prompt for context.

Your `process_refund` tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (5% of calls), and business errors ("Order exceeds day-return-window", "Item already refunded") that are permanent (12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.

What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- ☐ **A** Add a `check_refund_eligibility` tool that must be called before `process_refund` to prevent business rule violations.
- ☒ **B** Return structured error responses with `retriable: false` for business errors and a customer-friendly explanation for Claude to use.
- ☐ **C** Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.
- ☐ **D** Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.

During a billing dispute resolution, your agent successfully retrieves customer info via `get_customer` and order details via `lookup_order`, but when attempting to call `process_refund`, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

- ☐ **A** Implement automatic retries with exponential backoff for `process_refund`, keeping the conversation open until the refund is successfully processed
- ☐ **B** Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically
- ☒ **C** Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later
- ☐ **D** Escalate immediately to a human agent since the refund action cannot be completed

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

- ☐ **A** The verification tool is clearing the agent's internal state after each successful validation step.
- ☐ **B** The prompt lacks instructions telling Claude to remember information across multiple exchanges.
- ☒ **C** The conversation history isn't being passed in subsequent API requests.
- ☐ **D** Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.

You're implementing the escalation logic for when the agent should call `escalate_to_human`. Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

- ☐ **A** Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.
- ☐ **B** Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.
- ☒ **C** Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.
- ☐ **D** Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.

A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains `lookup_order` results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information. What approach most reliably handles returning customers?

- ☒ **A** Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.
- ☐ **B** Resume with full history but filter out previous `tool_result` messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.
- ☐ **C** Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.
- ☐ **D** Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness.

Your agent has called `lookup_order` multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history) and the outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before making additional lookups?

- ☐ **A** Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions
- ☒ **B** Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details
- ☐ **C** Proceed with additional lookups without modifying the existing tool output context
- ☐ **D** Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues

A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

- ☐ **A** Summarize earlier turns into a narrative description, preserving full message history only for the active issue.
- ☐ **B** Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.
- ☒ **C** Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.
- ☐ **D** Implement sliding window context that retains the most recent 30 turns.

When implementing your `lookup_order` MCP tool, the backend sometimes returns errors (e.g., "Order not found" or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

- ☐ **A** Return a success response with a "status" field indicating the error type
- ☒ **B** Return the error message in the tool result content with the `isError` flag set to true
- ☐ **C** Log the error server-side and return an empty result to avoid confusing the model
- ☐ **D** Throw an exception from the tool handler so the agent framework can catch and log it

Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order`, it identifies that the dispute involves a promotional pricing error requiring manager approval—beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

- ☐ **A** Call `escalate_to_human` passing only the customer's original message.
- ☐ **B** Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.
- ☐ **C** Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.
- ☒ **D** Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`.

Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent—this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

- ☐ **A** Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).
- ☐ **B** Strengthen the system prompt with emphatic language: "CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly."
- ☐ **C** Modify the refund tool to return an error with message "Amount exceeds policy limit—please escalate" when threshold is exceeded.
- ☒ **D** Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation.

A customer sends: "This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW." The agent has not yet called any tools to investigate the account. What should the agent do?

- ☐ **A** Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats their request.
- ☐ **B** Immediately call `escalate_to_human` with the conversation history.
- ☒ **C** Acknowledge the frustration and ask one targeted question to understand the specific issue before escalating.
- ☐ **D** First call `get_customer` and `lookup_order` to gather account context, then escalate to a human agent.

A customer writes: "I've been going back and forth on this return for days. I just want to speak to someone who can actually help me." The agent has confirmed via `lookup_order` that the return is straightforward—within policy and eligible for immediate processing. What should the agent do?

- ☐ **A** Ask what specifically hasn't worked in previous attempts before deciding whether to escalate or resolve automatically
- ☐ **B** Call `escalate_to_human` immediately to honor the customer's request
- ☒ **C** Acknowledge frustration, inform them this is resolvable now, and offer to complete it or escalate
- ☐ **D** Process the refund via `process_refund` to resolve the underlying issue, then inform them it's complete

When the agent calls `lookup_order` and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?

- ☐ A The agent executes the remaining steps in a tool sequence planned at the start of the request.
- ☒ B The order details are added to the conversation and the model reasons about which action to take.
- ☐ C The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.
- ☐ D The orchestration layer automatically routes to the next tool based on the order's status field.

After investigating a billing dispute over 25+ turns, you've identified that duplicate charges occurred due to a payment gateway timeout triggering retry logic. The required refund (\$847) exceeds your \$500 authorization limit. You need to call `escalate_to_human`, and the human agent won't have access to your conversation transcript. What context should you pass to enable effective resolution?

- ☐ A Your diagnosis and the refund amount only.
- ☒ B A structured summary: customer ID, root cause, refund amount, and recommended action.
- ☐ C The complete conversation transcript with all tool results.
- ☐ D The customer's original complaint verbatim plus the tool result excerpts showing duplicate transactions.

Production logs reveal inconsistent error handling: when `lookup_order` fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: `{"isError": true, "content": [{"type": "text", "text": "Operation failed"}]}`. The agent cannot distinguish between error types. What's the most effective improvement?

- ☒ **A** Enhance error responses with structured metadata: include `errorCategory` (transient/validation/permission), `isRetryable` boolean, and a description of what caused the failure.
- ☐ **B** Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.
- ☐ **C** Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.
- ☐ **D** Create an `analyze_error` MCP tool the agent calls after any failure to determine the error category and recommended action.